17-626: Requirements for Information Systems
Assignment 2: Requirements Analysis

Scenarios, goal modeling, use cases and user stories all describe methods to identify and document stakeholder needs, before analyzing those needs to reveal refinement and pre- and post-condition relations that establish the context for software operations. These techniques also support engineers in linking stakeholder actions to quality attributes to see how quality interacts with and derives from functional requirements that serve quality. In this assignment, students will extract requirements from interview transcripts and explore models to improve their comprehension of stakeholder needs by contextualizing those needs in early examples of system design.

Required Readings

- Read 7.1-7.3, 8.1-8.6, 8.8.1-8.8.5 in Requirements Engineering by Axel van Lamsweerde, Wiley, 2009.
- Chapters 8-10 in Advanced Use Case Modeling by Armour & Miller, Addison-Wesley, 2001.

Learning Objectives

- Identify the relationships between user stories, user goals and use cases obtained from multiple stakeholder perspectives
- Identify redundancy, inconsistency and conflict among stakeholder requirements and their perspectives
- Analyze requirements for completeness using goal types, goal categories, refinement heuristics and patterns

Assignment Deliverables

This is an individual assignment. In this assignment, you will practice organizing stakeholder needs expressed in interview transcripts into use stories, goal models and use cases. Complete the assignment in three steps:

Step 1. Extract goals from each of your assigned interview transcripts using the `goalmodeling` Python framework[1] and assign unique goal IDs "G-*n*" for *n* from 1 to *N* total goals. Trace the goals back to the original transcripts to ensure they are accurate reflections of the source text and not hallucinations. Review the transcripts and resolve any ambiguity or vague terms by translating these terms into your own interpretations, expressed using goal models. Finally, review each relationship for logical consistency. To be consistent, a child goal should answer "how" a parent goal is satisfied and a parent goal should answer "why" a child goal is satisfied.

---

[1] By standardizing your model visualization using the `goalmodeling` framework, you will save time and avoid losing points for syntax errors (wrong shapes or arrows). The instructions to download and use the framework are available at GitHub: https://github.com/cmu-relab/goalmodeling

Ensure that, if there are any conditional goals/actions, those are described using the standard format “if [condition]” in the goal text (see p. 266 in van Lamsweerde). Apply the refinement patterns and heuristics to the models to yield new goals. For any goals identified from ambiguity or vagueness, refine these goals using your own interpretation of the system context and stakeholder need. Be sure the lowest-level goals are realizable and can be monitored or controlled by an agent. Save the Python code to a file: goal_model_FIRST.py where FIRST is the capitalized first name of the stakeholder transcript from which the goals were extracted.

Step 2. Create a combined goal model by merging the individual models extracted from each transcript. Identify any redundancies, inconsistencies and conflicts. If you find a duplicate goal, manually merge the goals into one goal by re-writing the goal you plan to retain to include any information expressed in the other goal. Goal conflicts arise if a stakeholder provides inconsistent information, which may be due to unstated conditions, or if two stakeholders express conflicting needs. Save the combined goal model to a file: goal_model_combined.py

Step 3. Review your combined goal model to identify goals that describe critical processes in the system design. Document those processes separately using the use-case format, including a use case ID “UC-*n*” for the *n* from 1 to *N* total use cases, the pre- and post-conditions and any conditional or exceptional steps in the flow of events. Create a traceability matrix (a two-column table) that maps each use case ID to any matching Goal IDs. If steps in a use case match a goal, then trace the step to the goal using the step ID format “UC-*n*@*m*” for use case *n* and step *m*. If a use case or goal does not having a mapping, include them in your matrix with a “None” in the opposing column to indicate there is no mapping. Account for any goals or use case steps not stated in the transcript by highlighting these items in YELLOW highlight. This may include goals identified using the refinement patterns or your tacit knowledge.

Submit a ZIP archive that contains the following:

- Python code to build the individual and combined goal models using **goal_model_FIRST.py** and **goal_model_combined.py** conventions.
- Working Mermaid link to visualize combined goal model in a file, entitled **mermaid.txt**
- Use cases in a JSON file entitled **use_cases.json**, formatted using the schema in Appendix B.
- Traceability matrix in JSON file entitled **matrix.json**, formatted as a 2xP array for P equal to the number of rows, at most the total number of goals and use cases and use case steps, if nothing matched.
- One-half page in a PDF file entitled **reflections.pdf** that describes your experiencing extracting the goals from transcripts, merging the goal models, creating and analyzing the models and evaluating AI in the development of these artifacts. Discuss the advantages and disadvantages of applying the techniques.

Further Guidance

- When reviewing the goal model, examine the actions and identify high-level goals that frame those actions (e.g., why do the actors perform those actions, or what outcomes are the actors trying to achieve, cease, maintain or avoid (see H7 on p. 314 and P1 on p. 321.)
- Traverse your refinement relations from high- to low-level goals, asking yourself why and how these goals are satisfied, while remaining in the problem space (see H4 on p.313). Stay as close to the results of your interview but fill in any gaps that you might encounter by introducing new goals. Refinement from high-level goals can be used to identify alternative solutions, particularly when thinking about soft goals (see H6 on p. 313).
- To assist with refinement, try to apply each of the goal refinement heuristics and try to document one goal for each. Apply annotations to your goal model (see Figure 8.12, page 308) using the *goalmodeling* framework to identify the code for each refinement heuristic that was used to identify a given goal refinement (see Appendix A).
- Conduct a pass over the goal model by considering the six refinement patterns for each applicable goal in the model. If a pattern applies and is used to discover new goals, annotate the model using the *goalmodeling* framework to indicate the pattern code (see Appendix A) where the pattern was applied.
- Ensure that you have identified at least one behavioral goal, one soft goal, one functional goal and one non-functional goal, include at least one alternative OR-refinement, and document any conflicts discovered from your interviews.
- Goal models must be legible. If converting from SVG to PDF, PNG or another graphics format, be sure to increase the size of the image to ensure that we can read the text.
- While you can use AI to extract and generate the models, you may not use AI to write your reflection statement. The depth of your reflection depends on your connection to and understanding of the models. Over-reliance on AI can lead to shallow reflections, and you may be quizzed on your knowledge of your models.

Assignment Submission

Assignments are due on the day and time posted in the course website. Submit electronically at the course website. *This is an individual assignment.*

AI Assistance

AI may be used to extract goals from the transcripts and to express those goals in Python and to extract and express the use cases in JSON; however, students will be held responsible for the correct use of the notation and syntax and for any missing, inaccurate or hallucinated information. AI assistance is not permitted anywhere else in this assignment and any indication of using AI assistance beyond the scope this is permitted will be handled in accordance with the AI Assistance Policy in the course syllabus.

Evaluation Criteria

The assignment will be evaluated using the rubrics shown in the course website.

Appendix A

The following heuristic and pattern codes should be used to annotate `Refinement` objects in the goal model (see Section 8.8, pp. 309- in van Lamsweerde, 2009). Each refinement created by one of the heuristics and patterns should be labeled by the code, only.

Code	Refinement Heuristic
H3	Instantiate goal categories
H4	Ask “how” and “why” questions
H5	Split responsibilities
H6	Identify soft goals by analyzing pros and cons
H7	Identify agent wishes
H8	Analyze obstacles, threats and conflicts
H9	Check the converse of achievement goals
H10	Check the converse of conditional achievement goals
H11	Refine goals until they are assignable to single agents
H12	Abstract goals until the system’s boundary is reached

Code	Refinement Pattern
P1	The milestone-driven refinement pattern
P2	Case driven refinement pattern
P3	Decomposition-by-case pattern
P4	Guard-introduction pattern
P5	Divide-and-conquer pattern
P6	Unmonitorability-driven refinement pattern
P7	Uncontrollability-driven refinement pattern

Appendix B

JSON format for expressing use cases

Use cases are stored as a JSON list, wherein each use case is a dictionary with the `uc_name`, `uc_id`, `actors`, `pre_conditions`, `flow_of_events`, `post_conditions` and `non_functional`. Alternate flows can be included as a list assigned to the key `alternative_flows` or as integrated conditional logic by converting a step in the flow into a sublist with the first element as the event description, followed by the alternate flows for that event.

Example:

```
[
   {
      "uc_name": "Integrate Doctor's Goals",
      "uc_id": "UC-01",
      "actors": ["Doctor", "Patient"],
      "pre_conditions": [
         "The patient has authorized the doctor to provide exercise
            targets"],
      "flow_of_events": [
         "The doctor presents the recommended guidance based on the patient's
            age and mobility level",
         "The patient sets the frequency goal by the number of minutes per
            day, and number of days per week",
         [
            "The patient sets the intensity goal using the target heart rate
                range and heart rate maximum",
            "If the heart rate range or maximum exceeds the
                doctor's recommendations, then the patient will
                be asked to correct the entry"
         ]
      ],
      "post_conditions": ["The fitness frequency and/or intensity goals are
         stored and being monitored"],
      "non_functional": ["Unauthorized access attempts are periodically
         reviewed to identify persistent threats"]
   }
]
```